

DataEngineeringAndAI_ProjectPhase2.docx

 AUT University

Document Details

Submission ID

trn:oid:::20437:153609387

Submission Date

Oct 4, 2026, 5:29 PM GMT+13

Download Date

Oct 4, 2026, 5:33 PM GMT+13

File Name

DataEngineeringAndAI_ProjectPhase2.txt

File Size

42.1 KB

14 Pages

6,416 Words

36,557 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **48 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 2%  Publications
- 8%  Submitted works (Student Papers)

Match Groups

- 48 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 2% Publications
- 8% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	iieta.org	1%
2	Internet	assets-eu.researchsquare.com	<1%
3	Internet	export.arxiv.org	<1%
4	Internet	d197for5662m48.cloudfront.net	<1%
5	Internet	dev-journal.iainlangsa.ac.id	<1%
6	Submitted works	Trine University on 2026-09-19	<1%
7	Submitted works	Nexford Learning Solutions on 2026-10-03	<1%
8	Submitted works	The Kyoto College of Graduate Studies for Informatics on 2026-07-19	<1%
9	Submitted works	RMIT University on 2026-05-31	<1%
10	Submitted works	Yoobee on 2026-07-26	<1%

11	Submitted works	Erasmus University Rotterdam on 2026-08-12	<1%
12	Submitted works	University of Essex on 2026-08-21	<1%
13	Submitted works	Dublin Business School on 2026-01-07	<1%
14	Submitted works	Universidad TecMilenio on 2026-09-06	<1%
15	Submitted works	Southern New Hampshire University - Continuing Education on 2026-09-13	<1%
16	Internet	www.upgrad.com	<1%
17	Submitted works	Aston University on 2026-03-22	<1%
18	Submitted works	University College London on 2022-04-28	<1%
19	Submitted works	FACULTATEA DE MATEMATICA SI INFORMATICA on 2026-08-31	<1%
20	Submitted works	Mt San Antonio College on 2026-07-27	<1%
21	Submitted works	University of Greenwich on 2026-07-13	<1%
22	Internet	repository.sharif.edu	<1%
23	Submitted works	Hajee Mohammad Danesh Science And Technology University,Bangladesh on 202...	<1%
24	Submitted works	University of Essex on 2026-08-21	<1%

25	Submitted works	University of Wales Institute, Cardiff on 2026-08-03	<1%
26	Internet	espace.etsmtl.ca	<1%
27	Internet	monog.ufop.br	<1%
28	Internet	repository.president.ac.id	<1%
29	Submitted works	Colorado Technical University Online on 2026-09-28	<1%
30	Submitted works	Hogeschool van Amsterdam (HvA) - Tii on 2026-08-14	<1%
31	Submitted works	IUBH - Internationale Hochschule Bad Honnef-Bonn on 2025-06-01	<1%
32	Submitted works	Massey University on 2026-08-21	<1%
33	Submitted works	Massey University on 2026-08-23	<1%
34	Submitted works	Moodle2025 on 2026-07-22	<1%
35	Submitted works	The Kyoto College of Graduate Studies for Informatics on 2026-08-10	<1%
36	Submitted works	University of East London on 2026-09-01	<1%
37	Submitted works	University of Essex on 2026-08-19	<1%
38	Submitted works	University of Essex on 2026-08-19	<1%

39	Internet	
ojs.southfloridapublishing.com		<1%
40	Internet	
www.ixaeus		<1%
41	Submitted works	
Georgia Institute of Technology Main Campus on 2026-07-26		<1%
42	Submitted works	
Aston Online on 2026-08-11		<1%
43	Submitted works	
University of Hertfordshire on 2025-01-06		<1%

Task 1: Project Phase-I Summary

This project looks at credit card fraud detection, framed as a binary classification problem: given a set of transaction-level features, predict whether an individual transaction is fraudulent or legitimate. The target variable is Class, where 1 is fraud and 0 is a legitimate transaction.

The dataset used is the Credit Card Fraud Detection dataset, sourced from Kaggle and originally by the Machine Learning Group at Universite Libre de Bruxelles (ULB), covering transactions made by European cardholders over two days in September 2013. The dataset contains 284,807 records across 31 columns: 30 features (Time, V1-V28, and Amount) plus the target Class. V1-V28 are anonymised, PCA-transformed components with no disclosed real-world meaning, while Time and Amount are the only two features in their original form.

Phase 1 saw many key data-quality characteristics. The dataset contained 1,081 duplicate rows, which were removed to prevent data leakage between training and test sets. Amount was severely right-skewed (skewness of 16.979), which was addressed through a log transformation, reducing skewness to 0.161. No missing values, invalid values, or inconsistent entries were found. The most significant characteristic of the dataset is the severe class imbalance. Only 492 of the original 284,807 transactions (0.17%) are fraudulent, a proportion that stayed unchanged after cleaning (473 of 283,726, 0.167%).

This problem is suited to an LCS-based solution for many reasons. LCS systems evolve interpretable, human-readable rule sets rather than opaque decision boundaries, which is needed in a fraud-detection context where decisions may need to be explained to stakeholders such as banks or regulators. In addition, Phase 1's correlation analysis showed that only a few features (V17, V14, V12, V10, V11, V4) had linear signals for predicting fraud, whilst many features contribute little to nothing. That is a pattern LCS's rule discovery mechanism is suited to exploit, since it learns to condition rules on relevant feature ranges rather than needing all 30 features to contribute. The dataset's structure (fully numeric, tabular, binary-labelled) is also compatible with standard LCS implementations such as eLCS.

Task 2: Original LCS System on raw Dataset

Yaacoub:

Even though accuracy seems high (99.83%), this is very misleading given the class imbalance. The balanced accuracy of 0.5 and zero precision, recall, and F1-score confirm the original LCS system failed to detect any fraud in the raw, unprocessed data, motivating the preprocessing and improvements in Task 3 and 4.

Matthew:

An original, unmodified of eLCS, was applied to the raw (uncleaned and untransformed) dataset. To sub-sample instead of relying on train_test_split stratify parameter, it is an explicitly computed class specific sample sizes using the raw fraud rate of 0.1727%. Each class was sampled independently, then merged and shuffled, thus directly controlling the fraud rate of the sample rather than doing it indirectly to preserve class distribution.

The baseline used managed to get no detections. The Balanced Accuracy is at 0.5000, while precision, recall and F1-score are 0.0000. While the Accuracy score appears high at 99.82%, it is misleading as a degenerate

classifier simply outputting "legitimate" for every single input would have achieved near identical accuracy despite detecting 0 frauds.

Task 3: Data Preprocessing and Feature Engineering

Yaacoub: Missing values, invalid values, inconsistent categories, and data types were addressed in Phase 1, where none were found. The 1,081 duplicate rows identified in Phase 1 are removed here before splitting, to prevent identical transactions appearing in both **training and test sets**. The dataset is split into training and test sets before any scaling or resampling is applied, **ensuring no information from the test set influences preprocessing decisions**. Outliers in Amount, investigated using the IQR method in Phase 1, was retained rather than removed, as they represented genuine high-value transactions rather than invalid data, and fraud was only marginally overrepresented among them.

Matthew:

In phase 1, data quality was evaluated; it yielded 0 missing values, invalid values, inconsistent categories and data types. 1,081 duplicate row was then removed before the split to avoid the same transactions during both train and test sets. Also in phase 1, the Outliers were examined with Z-scores ($|z| > 3$) in Amount which revealed 4,063 transaction (1.43%). Only 11 of these 4,063 were frauds. This proportion is in line with the fraud rate of the dataset, implying that an exceptional transaction amount is not a good signal for a fraud. Thus, it was decided that outliers would not be removed because excluding them risks removing valid frauds.

Task 2's exact-per-class-count subsampling method was reused, keeping the original fraud rate and reducing to 40,000 rows. The split for training/testing was done right after sub-sampling and prior to any fitted preprocessing step to make sure test information would never contaminate the training phase.

Amount was strongly right-skewed; Yeo-Johnson power transformation (Yeo & Johnson, 2000) was used as it allows transactions of amount 0, a problem for the normal log transform. Additionally, a Robust Scaler was used to scale Time as its median/IQR-based scaling with the decision to keep rather than removing the outliers. Both transforms **were fitted only on the training set, and the subsequent test sets** were transformed with `.transform()`

The number of features were reduced from 30 to 10 by selecting the 10 features that had the highest absolute correlation with Class on the training set only. This reduction is important for eLCS because the rule population must search a match set covering all the chosen features. Including useless features adds unnecessary search space and decreases rule specificity.

The 10 chosen features were transformed in five quantile-based bins using `KBinsDiscretizer` fit on the training data only. This choice is supported by the fact that eLCS rules can be better combined with discrete, categorical-like conditions than arbitrary continuous sub-ranges. **SMOTE was then applied only to the training set**, synthesizing more fraud examples. This will result in significantly more examples of fraud for the rules discovered

Running the original eLCS after preprocessing, feature selection and discretization, but with the 1:1 SMOTE balancing applied, no hyperparameter changes reveal just preprocessing has some positive impact on the results (Recall jumps from 0.0000 to 0.9231). **But this increase in**

the accuracy will be mapped much stronger and hence the highly accurate rule will have higher fitness. Further, configuration flag `do_correct_set_subsumption` was enabled (if a rule newly inserted into the set can be covered/subsumed by an existing rule, it is not introduced in the population, and the set's covering rule receives an increment to its numerosity). Neither of these modifications require changing the LCS loop beyond simply changing configuration parameters from the previous tasks.

Metric	Task 2 (raw)	Task 3 (preprocessing only)	Task 4
(improved)	Accuracy	0.9982	0.9782
	0.9945		
Balanced Accuracy	0.5000	0.9507	0.9204
0.0000	0.0649	0.2075	Precision
0.9231	0.8462	Recall	0.0000
0.3333	F1-score	0.0000	0.1212
	Training time	36.33s	15.42s
			11.80s

The application of a partial SMOTE ratio combined with an increased `nu` and `correct-set` subsumption more than tripled precision to over 0.20 compared to Task 3 (0.0649 vs 0.2075) with a modest reduction in recall (0.9231 to 0.8462) and F1-score increased almost threefold (0.1212 to 0.3333). This pipeline iteration seemed to successfully balance fraud detection against excessive classification of legitimate transactions as fraudulent.

Task 5: Experiments

Yaacoub

Validation Strategy: An 80/20 stratified train-test split was used, rather than k-fold cross-validation mainly due to the computational cost of LCS training and prediction (single training runs took 40-75 seconds even on subsampled data (Tasks 2 and 4)) and generating predictions for the full 56,746-row test set took approximately 25 minutes per model. K-fold cross-validation would multiply this cost by the number of folds, making it impractical within the project's time constraints. Stratification confirmed the severe class imbalance (0.167% fraud) was preserved proportionally in both the training and test sets. Evaluation Metrics: Given the class imbalance seen in Phase 1, accuracy alone was deemed unsuitable, as it can appear high even when a model fails to detect any fraud (seen by the baseline's 99.83% accuracy despite zero recall). Balanced accuracy, precision, recall, and F1-score were prioritised, alongside ROC-AUC and PR-AUC, with emphasis on PR-AUC as the more informative metric on imbalanced data. Confusion matrices were also looked at, since they reveal the practical cost of each error type (missed fraud vs. false alarms) discussed as a key trade-off in Phase 1's motivation.

Metric	Baseline	Improved (30 features)	Confusion Matrix
	[[56648, 3], [95, 0]]	[[55740, 911], [14, 81]]	ROC-AUC
	0.6578	0.9656	PR-AUC
		0.2223	0.5834

The baseline model's confusion matrix confirms it failed to detect any of the 95 fraudulent transactions in the test set (0 true positives). The improved model correctly identified 81 of these 95 cases, at the cost of 911 false positives among legitimate transactions. ROC-AUC improved from 0.658 (barely above the 0.5 chance level) to 0.966, and PR-AUC, the more appropriate metric given the class imbalance, improved from 0.222 to 0.583.

Statistical Test: McNemar's test was used to compare the paired predictions of the baseline and improved models on the identical test set, since it is suited to comparing two classifiers correct/incorrect

outcomes on matched samples. The test produced a statistic of 81.0 and a p-value of 1.02×10^{-177} , far below the $\alpha = 0.05$ significance threshold. This provides strong statistical evidence that the improved system's performance differs majorly from the baseline, confirming that the observed gains are not attributable to chance.

Matthew:

Validation strategy: For this investigation, a simple 80/20 stratified train-test split was chosen rather than k-fold cross-validation. Two factors motivate this decision. First, and more practically, fitting an eLCS instance took up to around 36 seconds depending on parameter configuration. A k-fold cross-validation would increase that time k-fold, which, given the 6 model comparisons performed in Task 6, would likely be prohibitively slow.

The second, and more important, factor is the class imbalance. Even using only the 40,000-row subsample, there are only roughly 54-55 cases of fraud in the training set. Subdividing this already limited fraud class into k folds creates a high likelihood of certain folds containing essentially zero fraud for eLCS to train on, even with carefully chosen hyperparameters. A stratified split ensures all available fraud cases are effectively concentrated in one training set and one test set, and a fixed random_state=42 was used throughout for reproducibility.

Evaluation metrics: Previously, accuracy, balanced accuracy, precision, recall, and F1-score were reported for all of Tasks 2-4. Here, additional metrics are reported: ROC-AUC, PR-AUC, and the confusion matrix. ROC-AUC can be used to summarize the quality of the ranking over the full range of decision thresholds and might be perceived as too optimistic if class imbalance is high, since the false positive rate is estimated over a large set of legitimate transactions. PR-AUC, on the other hand, can be seen to be more sensitive to the performance on the minority class (fraud) and is thus considered a more informative ranking score here due to the actual fraud rate of 0.17%.

Metric	Task 3 (preprocessing only)	Task 4 (improved)	Confusion matrix
	[[7814, 173], [1, 12]]	[[7945, 42], [2, 11]]	ROC-AUC
	0.9259	0.9626	PR-AUC
		0.8771	0.8474

Task 4's confusion matrix shows far fewer false positives than Task 3 (42 vs 173), at the cost of one additional false negative (2 vs 1), consistent with the precision-for-a-little-recall trade described in Task 4. Task 4 also has a higher ROC-AUC (0.9626 vs 0.9259), though Task 3 has a marginally higher PR-AUC (0.8771 vs 0.8474), reflecting that Task 3's ranking captures slightly more of the precision-recall trade-off in raw area terms even though its actual 0.5-threshold predictions are much noisier.

Statistical test: McNemar's test: McNemar's test was used to check whether Task 4's improvement over Task 3 is statistically significant rather than noise from one particular test-set split. It is the appropriate choice here rather than a t-test, because both models were evaluated on the exact same test instances; it works directly on the disagreements between the two classifiers, where Task 3 was correct and Task 4 was wrong, and vice versa. The null hypothesis (H_0) is that Task 3 and Task 4 have the same error rate on this test set; the alternative

(H1) is that their error rates differ, tested at $\alpha = 0.05$.

Of the 8,000 test cases, Task 4 corrected 132 errors found by Task 3 (these are almost all false positives), whereas Task 3 was correct in only 2 cases where Task 4 was wrong, producing a very lopsided pattern of disagreement. The p-value from McNemar's exact test is $8.31e-37$, very much less than $\alpha = 0.05$, so the null hypothesis is rejected. This indicates that the difference between Task 4 and Task 3 is statistically significant on this test set and is unlikely to be explained by chance.

Task 6: Model Comparison

Yaacoub:

The proposed improved LCS-based system was compared against the original LCS on both raw and preprocessed data, and against three conventional non-deep-learning models (Random Forest, Logistic Regression, and Decision Tree), all trained on the same SMOTE-balanced training data and evaluated on the untouched, real-world-imbalanced test set.

Model	Training Time	Accuracy	Balanced Accuracy	Precision	Recall
F1	ROC-AUC	PR-AUC	Original LCS (raw)		39.41s
0.9983	0.5000	0.0000	0.0000	0.0000	0.6578
0.2223			Original LCS (pre-processed)	35.05s	0.9884
0.9154	0.1110	0.8421	0.1961	0.9668	0.5506
	Improved LCS (30 features)	76.97s	0.9837	0.9183	
0.0817	0.8526	0.1490	0.9656	0.5834	
Random Forest	409.02s	0.9995	0.8736	0.9221	
0.7474	0.8256	0.9691	0.8122	Logistic	
Regression	1.12s	0.9735	0.9237	0.0528	0.8737
0.0996	0.9592	0.6829	Decision Tree	49.62s	
0.9976	0.8464	0.3771	0.6947	0.4889	0.8464
0.2625					

The original LCS on raw data (Task 2) failed completely, with balanced accuracy at chance level (0.500) and zero recall. Applying preprocessing alone (SMOTE, scaling) to the original, unmodified LCS system produced an improvement, with balanced accuracy rising to 0.9154 and recall to 0.8421 which is nearly matching the fully improved system's performance (balanced accuracy 0.9183, recall 0.8526). This indicates that preprocessing, rather than the hyperparameter tuning applied in Task 4, was the primary force of performance improvement; the gain from tuning alone was small by comparison.

Among the conventional models, Random Forest was the standout performer, achieving the highest accuracy (0.9995), by far the best precision (0.9221), and the best ROC-AUC (0.9691) and PR-AUC (0.8122) of all six models tested, though at a substantially higher training cost (409.02 seconds) than the other conventional models. Logistic Regression achieved the highest recall of any model tested (0.8737) but the lowest precision (0.0528), reflecting its simple linear decision boundary combined with SMOTE-balanced training data, which biases it toward predicting fraud. Decision Tree performed moderately across all metrics, without excelling on any single one.

Overall, the LCS-based models (preprocessed and improved) achieved a similar profile to Logistic Regression – high recall, low precision – while Random Forest achieved a better balance between the two, and the best overall discriminative ability (PR-AUC). This suggests that, for this dataset, conventional methods such as Random Forest may offer superior raw predictive performance compared to LCS-based approaches, though LCS retains an advantage in interpretability through its evolved

rule sets.

Matthew:

Three conventional models were chosen to span different modelling assumptions rather than three variants of the same idea: **Random Forest** (an ensemble of decision trees, a strong standard tabular-fraud-detection baseline capturing non-linear feature interactions without much tuning), k-Nearest Neighbours (instance-based and non-parametric, an interesting contrast to LCS since both are, in different ways, similarity/matching-based approaches), and Gaussian Naive Bayes (fast and probabilistic, a simple low-cost baseline revealing whether the dataset needs anything more sophisticated than treating features independently). All three were trained on the same 10 correlation-selected features used by the LCS models, but in continuous, scaled form (prior to discretisation) with SMOTE reapplied, keeping the feature set identical across all six systems so differences reflect the modelling approach rather than a different data representation.

Model	Train Time	Acc.	Bal. Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
Original LCS (raw)	36.33s	0.0000	0.0000	0.0000	0.8084	0.2435	0.5000	
Original LCS (preprocessed)	15.42s	0.0649	0.9231	0.1212	0.9259	0.8771	0.9507	
Improved LCS	11.80s	0.8462	0.3333	0.9626	0.8474		0.2075	
Random Forest	14.41s	0.9991	0.9228	0.6875	0.8462	0.7586		
k-Nearest Neighbours	0.15s	0.9573	0.8419	0.8462	0.4400	0.9606	0.9965	
Gaussian Naive Bayes	0.02s	0.9214	0.2973	0.8462	0.9810	0.9137	0.5105	
		0.8462	0.1264	0.9302	0.1678		0.0683	

The original LCS on raw data failed almost completely at its default decision threshold, with balanced accuracy at chance level and zero recall, though its ROC-AUC (0.8084) shows some weak internal ranking signal that simply never crossed the 0.5 threshold given how few fraud examples it saw during training. Preprocessing alone fixed this dramatically, and the further tuning applied in Task 4 mainly traded some of that recall for a large gain in precision.

Among the conventional models, **Random Forest** was the clear standout, achieving the best precision (0.6875), the best F1-score (0.7586) of any model tested including all three LCS variants, and accuracy competitive with the improved LCS, all while matching the same recall (0.8462). k-Nearest Neighbours and Gaussian Naive Bayes both matched the 0.8462 recall of the improved LCS almost exactly (a product of the same SMOTE-balanced training data pushing every model toward similar sensitivity), but neither matched Random Forest's precision; Naive Bayes in particular performed worse than even the unmodified preprocessed LCS on precision and F1-score, consistent with its strong independence assumption being a poor fit for these PCA-derived, likely-correlated features. Overall, Random Forest outperformed every LCS variant tested on precision and F1-score, while achieving comparable recall and balanced accuracy. Training time does not favour Random Forest as clearly: it trains faster than the raw and preprocessed LCS baselines, but not faster than the already-tuned improved LCS, so its advantage over the strongest LCS variant is in predictive performance rather than speed. This suggests that for this specific dataset and feature set, a conventional ensemble

method can outperform LCS-based approaches on raw predictive performance. LCS's main advantage that a black-box ensemble like Random Forest cannot offer is direct interpretability through its evolved rule set.

Task 7: Interpretation of Results

Yaacoub:

Interpreting the Model Comparison: Task 6 showed that preprocessing was the primary force of improvement for LCS-based models, with the original LCS on preprocessed data almost matching the fully improved system's balanced accuracy and recall. Compared with conventional models, **Random Forest** achieved **the best overall precision-recall balance and** discriminative ability (PR-AUC = 0.8122), whilst LCS-based models shared a similar profile to Logistic Regression: high recall, low precision. This insinuates Random Forest may offer better raw predictive performance on this dataset, but LCS offers predictions that can be traced back to a specific, human-readable rule.

Example LCS Rules: Three rules were taken from the improved LCS system's final rule population (923 total rules), selected for high match counts and high accuracy:

Rule 1 (matched 4,443 times, 99.86% accuracy): predicts fraud when V2 is between -10.72 and 20.60, V6 between -14.78 and 5.20, V14 between -19.58 and -2.78, V18 between -9.67 and 0.28, V19 between -2.07 and 5.33, V23 between -10.18 and 7.53, V26 between -1.01 and 0.85, and V27 between -1.17 and 5.32.

Rule 2 (matched 4,513 times, 92.22% accuracy): predicts fraud when V12 is between -17.62 and -1.50, V16 between -22.60 and 3.60, V19 between -2.07 and 5.05, V21 between -16.38 and 17.17, V23 between -10.52 and 11.53, V24 between -2.10 and 2.41, and V28 between -5.46 and 5.38.

Rule 3 (matched 3,041 times, 99.31% accuracy): predicts fraud when V2 is between -24.07 and 19.17, V13 between -3.06 and 2.15, V14 between -20.18 and -3.24, V20 between -6.01 and 9.83, V25 between -0.99 and 1.92, and V26 between -1.42 and 2.16.

Discussion of Decision-Making Process: Each rule represents a combination of conditions across several features. A transaction is classified as fraudulent only if all its feature values fall within every specified range. V14 appears as a narrow, negative range in both Rules 1 and 3 (approximately -20 to -3), consistent with Phase 1's correlation analysis, where V14 (-0.293) was among the strongest negative correlation of fraud. V12 appears as a negative range in Rule 2, matching its Phase 1 correlation (-0.251). V19 and V23 recur across multiple top rules with narrow ranges despite showing negligible linear correlation with Class in Phase 1 (0.034 and -0.006 respectively), meaning LCS may be capturing non-linear or interaction-based patterns that a simple correlation coefficient would not detect.

Supporting Interpretability: Unlike Random Forest or Logistic Regression,

which make predictions through aggregated votes across many trees or one global weighted combination of all features respectively, each LCS rule is a readable "if-then" statement. A specific prediction can be explained by pointing to the exact rule (or rules) that matched the transaction and the exact feature ranges responsible which is a practical advantage in a fraud-detection context, where a flagged transaction may need to be explained to a customer, bank officer, or regulator.

Trustworthiness and Practical Meaningfulness: This advantage comes with an important issue. Because V1 through V28 are PCA-transformed components with no disclosed real-world meaning (Phase 1), a rule such as "V14 between -19.58 and -2.78" cannot be translated into a business-interpretable statement like "transactions from this merchant category" or "purchases above a certain amount at unusual hours." The rules are mathematically precise and traceable, but not semantically meaningful to a human stakeholder without access to the original, undisclosed features. This limits the value of LCS's interpretability advantage for this specific dataset – the explanation is sound, but the content of the explanation is not actionable in a real-world deployment sense.

Matthew:

Interpreting the main findings from the model comparison Task 6 compared six systems on the same held-out test set (8,000 transactions, 13 fraud): the raw-data eLCS, the preprocessed eLCS, the improved eLCS, and three conventional models trained on the same 10 features. The raw-data eLCS baseline never predicted fraud at all, despite a respectable ROC-AUC of 0.8084 – the model can rank fraud cases somewhat above legitimate ones internally, but with only 54 fraud examples and no class balancing, it never had enough incentive during rule evolution to commit to predicting the fraud class. Preprocessing and balancing fixed recall almost completely (0.9231) at the cost of precision (0.0649), because full 1:1 SMOTE balancing made the rule population treat fraud as roughly as common as legitimate transactions. Task 4's partial SMOTE plus a sharper fitness exponent and correct-set subsumption shifted that trade-off back toward precision (0.2075, more than tripled) while giving up only a little recall (0.8462). Against the conventional models, every one matches the improved eLCS on recall (0.8462), and Random Forest is well ahead on precision (0.6875 vs 0.2075) and F1 (0.7586 vs 0.3333). Training time does not follow the same pattern: Random Forest (14.41s) is faster than the raw and preprocessed LCS baselines (36.33s, 15.42s) but is not faster than the already-tuned improved LCS (11.80s), so its real advantage over the strongest LCS variant is predictive performance rather than speed. This is an expected outcome rather than a failure of the LCS approach specifically: ensemble tree methods are well suited to tabular, already-engineered features like PCA components, whereas eLCS's rule evolution must additionally discover useful attribute combinations from scratch through covering, mutation and crossover. The trade-off this project is interested in is predictive performance versus interpretability, which is why the improved eLCS model's rule population is examined directly below.

Extracting the rule population

skeLCS's eLCS exposes the trained rule population through `export_final_rule_population()`, which writes every macro-classifier to CSV with the real feature names substituted in. Each row records which attributes the rule tests, the condition on each specified attribute, the

predicted class, and quality indicators: Accuracy, Numerosity, Correct Count, Match Count, and Specificity. The final population contained 673 macro-classifiers (424 predicting legitimate, 249 predicting fraud). Rules were filtered to those with Correct Count ≥ 10 , leaving 557 rules (373 legitimate, 184 fraud) with enough training evidence to be a genuine pattern rather than noise.

Three sample rules and how they drive predictions

The 10 features going into eLCS are quantile bin indices (0 = lowest 20% of training-set values, up to 4 = highest 20%) rather than raw PCA values. Because SMOTE interpolation after discretisation produces fractional bin values, eLCS ends up treating these attributes as continuous rather than discrete, representing each rule condition as a numeric range rather than an exact bin match; some raw bounds fall outside the true 0-4 domain due to eLCS's covering mechanism, so each bound is clipped to [0, 4] before being described below.

Rule 1 (broad, legitimate-transaction): IF V17 in bin-range [0.80, 4.00] THEN predict Legitimate. Accuracy 0.8930, correct on 5,031 of 5,634 matches. V17 was the single strongest fraud indicator among the 10 selected features by correlation (ranked 1st); fraudulent transactions in this dataset tend to take unusually low raw V17 values, which is exactly the bottom bin this rule excludes.

Rule 2 (two-attribute, fraud): IF V10 in bin-range [0.00, 0.78] AND V11 in bin-range [1.73, 4.00] THEN predict Fraud. Accuracy 0.7977, correct on 2,828 of 3,545 matches. V10 and V11 rank 5th and 8th by correlation among the 10 selected features - weaker individually than V17 or V14, so this rule relies on combining two moderately-informative signals rather than one dominant one, which shows up as a noticeably lower accuracy than Rule 1.

Rule 3 (three-attribute, fraud): IF V17 in bin-range [0.00, 1.12] AND V14 in bin-range [0.00, 0.66] AND V11 in bin-range [3.38, 4.00] THEN predict Fraud. Accuracy 0.9841, correct on 2,470 of 2,510 matches. This rule is both more specific (three attributes) and more accurate than Rules 1 and 2, reusing V17 and V11 alongside V14 (ranked 2nd by correlation) - all three independently known to carry fraud signal. This is the general pattern in the population: higher-specificity rules (3-4 attributes) reach higher accuracy than broader 1-2 attribute rules, at the cost of covering fewer training instances.

How Rule-base outputs support interpretability

Every eLCS prediction is supported by the rule(s) that went active for that one instance. Each rule has a short, human-readable conjunction of feature-range conditions, and its own track record, in terms of accuracy, numerosity, and match count. This structure is different from Random Forest, which generates a prediction by averaging hundreds of small paths through the forest of 100 trees. In Random Forest, there is no single, concise rule to explain why a fraud prediction was made for that transaction without a post-hoc explainability tool like SHAP. With eLCS, the explanation comes built in with the prediction, not with an added explainability step. Also, the rules are rated based on their inherent reliability statistics as opposed to being taken as equally trustworthy. For example, Rule 1 has a Correct Count in the thousands and high accuracy (0.89), so can reasonably be considered a real pattern. However, a rule seen only once or twice should not be treated as a reliable pattern, and the model distinguishes between them using these fields without needing to run an explainability test.

Are these explanations trustworthy and practically meaningful?

Yes, to a point, and with caveats. The evidence for belief is twofold: first, each of the three rules holds against thousands of training matches, and second, the features mentioned in those rules (V17, V14, V10, V11) were independently deemed the most predictive in Task 3. As described there, V17 and V14 were the top-two strongest correlates of fraud among all candidate features, and V10 and V11 held 5th and 8th place, respectively.

That these rules lean on top-performing attributes for detecting fraud suggests the rule population is converging on real predictive patterns rather than spurious connections (though it's still worth noting that Rule 2, the two-attribute rule, incorporates a weaker pair of inputs than Rules 1 and 3).

All three are also directionally consistent in their fraud correlation (V17-low, V14-low, V10-low, V11-high).

The caveats are threefold. First, because the entire input pool for the fraud class is limited - only 54 instances at the subsample count where training occurs - the absolute match counts are amplified via SMOTE (as SMOTE synthesizes fraudulent instances through neighbour interpolation). While this doesn't invalidate the pattern match, it means a count of, e.g., 5,000 matches relies on what those 54 original cases define, plus their SMOTE-neighbours, not an absolute diversity of 5,000 empirical ground-truth fraud cases.

Second, all features V1 through V28 are anonymised PCA components; thus, even if one is entirely convinced of the accuracy of Rule 1 (a low V17 being highly indicative of fraud), it is of little real-world actionability to a fraud analyst since the true semantic meaning of V17 remains undisclosed.

Third, and for reasons of both SMOTE's post-discretisation interpolation and eLCS's covering mechanism, one should read these rules as identifying bins rather than establishing an unbreakable physical barrier on the data range.

Summing these up, the rule population can reasonably be treated as a high-level descriptor of fraud behaviour in the PCA space, and is arguably more human-interpretable than the mechanisms of the Random Forest model, even if Random Forest is the objectively stronger prediction tool as demonstrated in Task 6. However, to transform these into hard, actionable operational thresholds for a fraud analyst, one would ideally need both the original, non-anonymised attribute names and a larger set of non-synthetic ground-truth fraud samples.

Task 8: Discussion, Limitations, and Future Work

Reasons for Performance Improvement: The improvement from the original LCS baseline (recall = 0.00) to the preprocessed and improved variants (recall > 0.83) was driven mainly by addressing the class imbalance using SMOTE, rather than by the hyperparameter tuning applied in the improved system. The original LCS on preprocessed data achieved balanced accuracy and recall almost identical to the fully improved system, meaning that eLCS's rule-discovery process was the main bottleneck: with only 378 fraud examples in the raw training set, the algorithm had a small amount of minority-class evidence to evolve rules covering fraud, regardless of how many learning iterations were allowed. This tells us that for genetic/evolutionary rule-based systems like LCS, data-level interventions (balancing class representation) may matter more than algorithm-level tuning when the imbalance is severe. This phenomenon was

experienced similarly between the two pipelines independently: for instance, for the second pipeline (40,000-row subset), the application of SMOTE (without a subsequent hyperparameter change) pushed the recall from 0.0000 to 0.9231 directly.

Practical Meaning of the Results: Beyond the metrics, the improved system's practical value lies in the trade-off it represents, that being catching roughly 85% of fraudulent transactions **at the cost of a substantially higher false-positive rate** (911 false alarms among legitimate transactions) than Random Forest's higher-precision approach. In a real deployment, this trade-off would need to be weighed against the business cost of each error type. A missed fraud case may cost more in direct financial loss than a false alarm cost in customer inconvenience, but too high a false-positive rate risks eroding customer trust and needing more manual review teams. The LCS-based system's interpretability, shown through its evolved rules in Task 7, offers an advantage for justifying individual decisions, even though its predictive performance did not beat Random Forest's. Two different points on the precision-recall trade-off spectrum were reached by the two pipelines examined here. The 40,000-row pipeline correctly captured about 85% of fraud (11 cases of fraud out of 13) while producing 42 false positives from the 7,987 legitimate transactions, a considerably tighter trade-off compared to the 911 false positives that the 15,000-row pipeline achieved (this comparison makes the sensitivity of this trade-off quite obvious, as it varies significantly with both the chosen SMOTE ratio and the chosen subset of training data).

Limitations: Many limitations affect the reliability and generalisability of these findings. Firstly, due to the high computational cost of LCS training and prediction, models were trained and evaluated on subsamples (15,000-40,000 rows, depending on the pipeline, for training) rather than the full dataset, which may not properly encapsulate the diversity of fraud patterns present in the complete 284,807-row dataset. Secondly, the severe class imbalance, while addressed through SMOTE, means synthetic fraud examples were generated through interpolation rather than representing new real-world fraud cases, which will not fully reflect the diversity of fraud tactics. Thirdly, the dataset's PCA-anonymisation, while needed for privacy, limits the practical interpretability of LCS's rule-based explanations, since feature conditions can't be mapped to real-world transactions. Finally, the dataset covers only a 48-hour window from September 2013, raising questions about whether patterns learned remain relevant to current fraud tactics, which always evolve.

Future Work: Many ideas could better this work. Testing alternative LCS variants (e.g., XCS, which uses accuracy-based fitness rather than eLCS's strength-based approach) could show whether rule-discovery improvements generalise beyond eLCS specifically. Alternative class-imbalance techniques, such as cost-sensitive learning or ensemble-based resampling methods (e.g., SMOTE-ENN), could be compared against the SMOTE approach used here to decide whether further gains in precision are achievable without sacrificing recall. External validation on a separate, more recent fraud dataset would help assess whether the evolved rules generalise beyond this specific dataset and timeframe. Finally, domain expert review. Essentially involving access to the original, non-anonymised transaction features which would be needed to properly decide whether LCS's rule-based explanations are actionable for fraud analysts in practice, addressing the interpretability limitation seen in Task 7.

References

- 1 Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- 2 Lemaitre, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(1), 559-563. <https://dl.acm.org/doi/10.5555/3122009.3122026>
- 3 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://dl.acm.org/doi/10.5555/1953048.2078195>
- 4 UrbsLab/scikit-eLCS (Version 1.2.4) [Computer software]. (2022, May 12). GitHub. <https://github.com/UrbsLab/scikit-eLCS>
- 5 Urbanowicz, R. J., & Moore, J. H. (2015). ExSTraCS 2.0: Description and evaluation of a scalable learning classifier system. *Evolutionary Intelligence*, 8, 89-116. <https://doi.org/10.1007/s12065-015-0128-8>
- 6 Yeo, I.-K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954-959. <https://doi.org/10.1093/biomet/87.4.954>
- Machine Learning Group - ULB. (2017). Credit Card Fraud Detection (Version 3) [Data set]. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Contribution statement

Yaacoub: I created the word document, structure, and worked on Task 1 just very quickly because it was easy. After that, me and Matthew worked individually from Task 2-7 to get our results, where in Task 8, we both contributed to writing the final discussion, our limitations, and the possible future work. I did a quick check of everything as well to make sure we didn't miss anything. Honestly, this was a fun project to work on with Matthew, he is a good teammate, and the only thing I can say that we could've changed is our choice of dataset considering the issues that we came across with the class imbalance. Otherwise, very good project, very good teammate, and a great outcome.

Matthew: For Task 2-7, Yaacoub and I built and ran our own LCS pipeline, from the baseline eLCS models to our improved ones. For me, this involved the leakage-safe preprocessing pipeline, the partial-SMOTE and hyperparameter optimisation and the McNemar's test to ensure the improvement was statistically significant and comparing it to the standard models and interpreting the resulting final set of rules. I collaborated with Yaacoub for Task 8 to write the discussion, limitations and future work. Overall, a great project to work on with Yaacoub, he was a good group mate and easy to work with. If I were to do anything differently it would probably to use a different dataset. The class

imbalance made essentially every part of the pipeline significantly more difficult, but it also proved to be the most compelling part to resolve as well.